



v Publications using TU Darmstadt's Corporate Design (TUDaCI)-0 <00

A Learned Advisor for Automated Materialized View Selection

Master thesis by Michael Girges

Date of submission: November 29, 2025

1. Review: Prof. Dr. Carsten Binnig
 2. Review: Johannes Wehrstein
 3. Review: Roman Heinrich
Darmstadt
-



TECHNISCHE
UNIVERSITÄT
DARMSTADT



Computer Science
Department
Data Management Lab

Erklärung zur Abschlussarbeit gemäß §22 Abs. 7 und §23 Abs. 7 APB der TU Darmstadt

Hiermit versichere ich, Michael Girges, die vorliegende Masterarbeit ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt zu haben. Alle Stellen, die Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Fall eines Plagiats (§38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei der abgegebenen Thesis stimmen die schriftliche und die zur Archivierung eingereichte elektronische Fassung gemäß §23 Abs. 7 APB überein.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, 29. November 2025

Signature

Abstract

Materialized View (MV)s are commonly used in databases to accelerate read queries, but their benefits come with high maintenance costs and full refreshes to avoid stale data. Incremental Materialized View (IMV)s offer an alternative by updating views incrementally through triggers. However, they perform best in read-heavy workloads and may introduce significant write-time overhead. This work proposes a machine-learning-based advisor that, given a SQL workload, predicts whether to use MV, IMV, or no views at all for optimal performance. The advisor jointly considers the cost of maintaining views during writes and the potential performance gains during reads. For this purpose, this work discusses the creation of a decision-tree-based cost models to predict the cost of using each view mode. Our approach aims to guide database administrators and automated tuning systems in selecting the most effective view strategy based on workload characteristics.

Contents

List of Figures	6
List of Abbreviations	7
1. Introduction	8
1.1. Context and Motivation	8
1.2. Problem Statement	8
1.3. Goals and Contributions	9
1.4. Thesis Outline	9
2. Background	10
2.1. Automated View Selection	10
2.2. Query Performance Prediction Using Cost Models	11
2.2.1. Tuple Time Tree (T3) Cost Model	12
2.3. Workload Generation	12
2.4. Incremental View Maintenance and pg_ivm	12
2.4.1. Incremental View Maintenance (IVM)	13
2.4.2. pg_ivm Implementation	13
3. Methodology	15
3.1. Training Data Curation	15
3.1.1. Dataset and Workload Generation	15
3.1.2. Experiment Data Collection	16
3.1.3. Feature Extraction from Execution Plans	17
3.2. Model Training	18
3.2.1. Training the T3-style Cost Model	18
3.2.2. Per-Pipeline Prediction Target	18
3.2.3. View Advisor Setup	19
3.3. Summary	20
4. Evaluation	21
4.1. Experimental setup	21
5. Conclusion and Future Work	22
5.1. Conclusion	22
5.2. Future Work	22
Bibliography	23

Appendices	25
A. Experiment Parameters	25

List of Figures

2.1. pg_ivm Trigger Mechanism for Incremental View Maintenance	14
3.1. Structure of a Single Experiment in the Workload	16
3.2. Structure of the Entire Workload Composed of Multiple Experiments	17
3.3. View Advisor Cost-Benefit Analysis Flow	20



List of Abbreviations

API Application Programming Interface

DBMS Database Management System

MV Materialized View

IMV Incremental Materialized View

IVM Incremental View Maintenance

DML Data Manipulation Language

ML Machine Learning

ILP Integer Linear Programming

DTA Database Tuning Advisor

T3 Tuple Time Tree

QEP Query Execution Plan

1. Introduction

1.1. Context and Motivation

MVs are a common optimization in database systems: by precomputing and storing query results they can significantly accelerate read queries. However, materialized views introduce maintenance costs whenever base tables change, because the view contents must be kept consistent with the underlying data. The naive strategy of full recomputation can become prohibitively expensive for workloads with frequent updates.

Incrementally maintained materialized views (IMVs) provide an alternative maintenance strategy by applying only the changes (deltas) caused by updates to base tables, instead of recomputing entire views. IMVs can significantly reduce maintenance work for many workloads, but they are not a universal win: their benefit depends on workload characteristics. In particular, IMVs tend to be most advantageous for read-heavy workloads with relatively small or structured updates, while write-heavy workloads can cause high incremental maintenance overhead and complex update propagation.

Choosing between no views, conventional MVs, and IMVs therefore requires trading off read-performance gains against the cost of maintaining views under updates. Manually exploring this trade-off is time-consuming and error-prone. This thesis proposes to automate that decision by building a machine learning-based advisor that, given a SQL workload composed of SELECT and Data Manipulation Language (DML) statements, recommends the view strategy that is likely to be optimal in terms of overall system performance and maintenance cost.

1.2. Problem Statement

The central problem addressed in this work is:

Given the estimated execution plans of a SQL workload (a sequence of SELECT and DML statements), determine whether the workload is optimally served by (i) no views, (ii) MVs with full recomputation, or (iii) IMVs with incremental maintenance, taking into account both query performance and view maintenance costs.

Concretely, this requires (a) modelling the costs and benefits of each strategy for a given workload, and (b) designing a predictor that maps workload characteristics to the recommended strategy. To support cost-aware decision making, this thesis includes the design of a view-refresh-cost estimation model inspired by decision-tree approaches (cf. the T3 system [15]).

This thesis answers the following research questions:

-
1. When is it beneficial to use MV, IMV, or no materialized view at all for a given SQL workload?
 2. Can a machine learning model accurately and efficiently predict the optimal view strategy for a workload based on its read and write characteristics?

1.3. Goals and Contributions

The goals of this thesis are to design, implement, and evaluate an automated advisor that recommends one of the three view modes (no view, MV, or IMV) for a given SQL workload. The primary contributions are:

- A cost estimation component for view refresh operations that informs the advisor; this component leverages a decision-tree-based approach inspired by the T3 system[15].
- The design of a machine learning-based advisor that takes workload features as input and outputs a recommended view mode (no view / MV / IMV).
- An empirical evaluation demonstrating the advisor’s effectiveness at selecting strategies that balance read performance and maintenance overhead across a variety of workloads.

1.4. Thesis Outline

The remainder of the thesis is organized as follows:

- Chapter 2 presents background material and related work on materialized views, incremental maintenance techniques, workload-aware tuning approaches and cost estimation methods.
- Chapter 3 outlines the overall methodology, including data collection, feature engineering, view refresh modelling procedures and view advisor design.
- Chapter 4 reports the experimental evaluation of both the view-refresh cost model and the proposed view advisor. The evaluation includes setup, metrics, results, and analysis.
- Chapter 5 concludes the thesis, summarizes findings, and outlines directions for future work.

2. Background

2.1. Automated View Selection

Automated database tuning, notably the selection and management of physical design structures such as MVs and indexes, has long been a critical and challenging task for database administrators. Traditional approaches often rely on manual expertise [23] or on complex rule-based and enumeration-based heuristics, which become particularly cumbersome for rich MV structures or complex queries [1].

Early automated MV selection work explored heuristic systems that choose candidate views based on estimated cost and validate them against the actual database [1]. Commercial systems like Microsoft SQL Server’s Database Tuning Advisor (DTA) provide workload-driven recommendations for indexes and MVs [1]. Other research formulates view selection as an optimization problem (for example, Integer Linear Programming (ILP)) [23]; while these formulations can be precise, they are often computationally expensive for large sets of subqueries or extensive workloads, and iterative methods may not guarantee convergence.

More recently, Machine Learning (ML) methods have been applied to a variety of database tasks with the goal of replacing handcrafted components by learned models that can automatically handle complexity and variability [7, 10]. A common paradigm in learned database components is workload-driven learning, which typically requires executing representative workloads on the target database to collect training data. Although effective in many settings, this data-collection step is costly and must often be repeated for each new database or substantially changed workload [10]. To address this limitation, zero-shot learning approaches have emerged: models are pretrained once and designed to generalize to unseen databases or workloads without per-database retraining. Initial zero-shot work has focused on tasks such as cost estimation [18], and researchers are investigating extensions of this paradigm to tasks like MV selection.

The maintenance of MVs - especially incremental maintenance - is another important axis in the literature. For example, commercial systems include refresh/maintenance modules that track refresh methods and schedules; some research and product directions explore using ML to predict refresh behavior or maintenance cost and to incorporate those predictions into recommendation logic [1]. This line of work suggests that predicted maintenance cost (including incremental refresh cost) can and should influence which views are recommended. The interaction between specific maintenance strategies (e.g., incremental refresh) and ML-based recommendation processes remains an active area where further research can provide valuable insights.

2.2. Query Performance Prediction Using Cost Models

Accurate query performance prediction is essential for effective physical design tuning, including MV selection. Traditional cost models, such as those used in query optimizers, estimate query execution time based on factors like I/O costs, CPU costs, and network costs [16]. However, these models often struggle to capture the complexities of modern database systems and workloads, leading to inaccuracies in performance predictions. The first cost model was proposed by Listgarten et al. [12] as part of their work on main memory databases. Moreover, a framework for building cost models for hierarchical database systems was introduced by Manegold et al. [13].

In the last couple of decades, learning-based techniques have been increasingly applied to the problem of query performance prediction. Gupta et al. pioneered the use of machine learning for query performance prediction by introducing a decision tree-based approach to predict query execution time ranges [5]. Query performance prediction has also been approached for static and dynamic workloads using regression models [2], who implemented these techniques on top of PostgreSQL. Calibration of existing cost models in PostgreSQL has been explored by Wu et al. [19], where they used small calibration workloads to adjust the cost model parameters. In combination with an analytical model, they were able to predict the query execution time for larger workloads [20]. This approach proved competitive to other contemporary learning-based methods.

Neural networks and deep learning techniques have also been employed for query performance prediction through feeding the tree-structured query plans into neural network architectures. For instance, Sun et al. [18] concluded that combining tree nodes is effectively handled using a pooling architecture. Zhou et al. proposed encoding single and multiple concurrent queries using a graph embedding approach [25]. This approach captures the interactions between concurrent queries and enables accurate predictions of execution times of whole workloads. Additionally, transformer-based architectures and augmenting the input data with database statistics have been shown to further improve prediction accuracy [24].

Since workload-based models do not work well in zero-shot settings, recent research has focused on developing zero-shot query performance prediction models. The first zero-shot query performance prediction model was proposed by Hilprecht et al. [6]. Their zero-shot model was trained and evaluated on different databases and workloads, demonstrating the ability to generalize to unseen databases without retraining. More recently, Wu et al. [21] introduced a zero-shot model that is trained on the complete Amazon Redshift [3] workload. They propose a hierarchical model in an effort to improve latency.

Learning-based approaches have shown promising results in query performance prediction, but they are often criticized in literature for their high end-to-end latency. This latency can be a significant drawback in scenarios where quick predictions are essential for decision-making. Lehmann et al. [11] commented on this issue by concluding that learning-based query optimizers have high optimization times, which renders them impractical for real-world usage. One approach to mitigate this issue is proposed by Wu et al. [21], where they use a query cache with a fallback to a local decision tree model to reduce the end-to-end latency of their zero-shot model.

Other non-learning-based approaches have been proposed to solve the query performance prediction problem. Since these approaches are not dependent on neural networks, they can be more interpretable and can be quicker to train and evaluate. For instance, cost models based on differential programming have been proposed by Hilprecht et al. [8]. They depend on learning some of the constants in

pre-existing cost model functions. Another approach is to use different groups of calibration constants for the fixed cost functions in the PostgreSQL cost model [22].

2.2.1. Tuple Time Tree (T3) Cost Model

Rieger et al. proposed the Tuple Time Tree (T3) cost model [15], which bridges the gap between high-accuracy, high-latency learning-based models and their lower-accuracy, low-latency counterparts. T3 is a hybrid cost model that combines a low-latency decision tree with pipeline-based query plan presentation and tuple-centric prediction targets.

The pipeline-based query plan representation divides the query plan into multiple pipelines, where each pipeline consists of a sequence of operators that can be executed without materializing intermediate results. This representation allows T3 to generate a flat feature vector for each pipeline and predicts its execution time individually. The total query execution time is calculated as the sum of all pipeline predictions.

On the other hand, tuple-centric prediction is based on the idea that, instead of predicting the total execution time for a pipeline, T3 predicts the expected time it takes to process a single tuple within that pipeline. Then, the per-tuple time is multiplied by the pipeline's input cardinality to estimate the total execution time of the pipeline. This approach is highly compatible with the decision tree architecture and is highly-scalable.

Furthermore, T3 improves inference time by compiling the trained model directly into native machine code using the `l1eaves` framework. This step drastically reduces the average prediction latency from 22 μ s in interpreted trees and 1ms in zero-shot neural networks down to 4 microseconds.

2.3. Workload Generation

In this work, we require a diverse set of training and evaluation workloads to train and assess our proposed models. Workload generation is a well-researched area, with various methods proposed to create synthetic workloads that mimic real-world query patterns. For example, Kipf et al. [9] introduced a method for generating synthetic SQL queries based on schema information and sampling values from the underlying data. Hilprecht et al. [6] built on this approach by enabling the generation of more complex queries that might include joins, random index creation and predicates with null comparison, regex-based string comparison and disjunctions. Additionally, in the Master's thesis "Caching Strategies based on realistic Workloads," Martin Stemmer [17] investigates the practical, cost-based trade-off between full result caching and incremental updates in modern data warehouses.

2.4. Incremental View Maintenance and `pg_ivm`

While MVs significantly accelerate analytical queries by precomputing results, they introduce the overhead of view maintenance. As the underlying base tables are modified (inserted, updated, or deleted), the MV becomes stale. Traditional maintenance strategies in systems like PostgreSQL rely on a full refresh strategy, which recomputes the entire view definition from scratch. While simple to

implement, full refresh is computationally expensive and inefficient for large datasets where only a small fraction of the data has changed.

2.4.1. Incremental View Maintenance (IVM)

Incremental View Maintenance (IVM) addresses the inefficiency of full refreshes by computing and propagating only the changes needed to bring the view up to date. Formally, given a view V derived from base relations $R_1, R_2, \dots, R_n$ via a query function Q , and a set of changes ΔR_i applied to the base relations, IVM seeks to compute ΔV such that the new view state V' is:

$$V' = V \cup \Delta V^+ \setminus \Delta V^-$$

where ΔV^+ represents tuples to be added and ΔV^- represents tuples to be removed.

For views involving simple selection and projection, the delta calculation is straightforward. However, for views involving joins and aggregations, the maintenance logic becomes more complex. For joins, the distributive property is typically utilized. If $V = R \bowtie S$, and R changes by ΔR , the change to the view is calculated as $\Delta V = \Delta R \bowtie S$. For aggregations, IVM typically employs a counting algorithm [4]. To handle deletions correctly in a view with grouped aggregates (e.g., 'GROUP BY'), the system must track the multiplicity (count) of each group. When a tuple is deleted from the base table, the count for the corresponding group in the view is decremented; the group is removed from the view only when the count reaches zero.

2.4.2. pg_ivm Implementation

Standard PostgreSQL does not support IVM natively; it requires extensions or manual trigger-based setups. The `pg_ivm` extension [14] is a state-of-the-art solution that brings IVM capabilities to PostgreSQL. Unlike asynchronous approaches that might use logs to defer updates, `pg_ivm` implements immediate maintenance, where the view is updated synchronously within the same transaction that modifies the base table.

`pg_ivm` operates by automatically generating and attaching triggers to the base tables involved in the MV definition. When a DML operation occurs, two trigger operations are executed, as illustrated in Figure 2.1:

1. **Delta Calculation:** The triggers capture the transition tuples (old and new values) and compute the corresponding delta for the view using the differential update logic described in Section 2.4.1. The computed ΔV is stored in temporary structures local to the DML transaction.
2. **View Update:** The changes calculated by the first trigger are applied to the materialized view to keep it consistent with the base tables.

To support general SQL queries including joins and aggregations without requiring unique keys on all projected columns, `pg_ivm` maintains the view using "counting" logic similar to the classic Counting Algorithm. This ensures that views correctly reflect the underlying data multiplicity, particularly for 'SUM' and 'COUNT' aggregates.

It is also worth noting that, while `pg_ivm` drastically reduces the cost of reading fresh data compared to a full refresh, it imposes a write penalty on the base tables. Every insert or update to the base

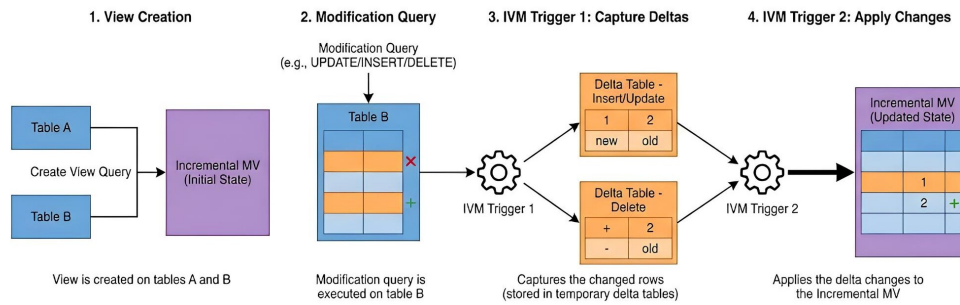


Figure 2.1.: `pg_ivm` Trigger Mechanism for Incremental View Maintenance

table now incurs the additional cost of computing ΔV and updating the MV index and storage. This trade-off—trading write latency for read consistency without full recomputation—makes `pg_ivm` a critical component in modern physical design tuning, as it alters the cost equations used by automated view selection algorithms. In some cases where the number of changed rows is a large portion of the total table cardinality, the incremental maintenance cost might be larger than a full refresh’s cost. Thus, `pg_ivm` provides a toggle to enable or disable IVM on a per-view basis, allowing users to choose between immediate incremental maintenance and traditional full refresh strategies based on their workload characteristics and performance requirements.

3. Methodology

This chapter details the methodology used to build the ML-based zero-shot view advisor. The primary objective is to create an advisor that can predict the optimal view strategy (no view, MV, or IMV) for a given SQL workload using execution time predictions of SELECT and DML execution plans from specialized T3 cost models. This is achieved by first curating a comprehensive training dataset by executing workloads and capturing their performance, then using this dataset to train a predictive cost model. The trained model is then integrated into a view advisor framework that analyzes workloads and recommends the best view configuration based on predicted costs and benefits.

3.1. Training Data Curation

The foundation of the ML advisor is a labeled dataset of query execution times. This dataset must capture performance metrics for both SELECT and DML statements in the IMV configuration. The phases mentioned below are repeated for 19 out of the 20 databases in the Amazon Redshift dataset [3], with the IMDB database being excluded to be used in evaluating the zero-shot model.

3.1.1. Dataset and Workload Generation

To build a robust model, we require a diverse set of training and evaluation workloads. We generate synthetic workloads that mimic the characteristics of real-world scenarios, such as the query distributions and read/write mixes found in datasets like the Amazon Redshift workload [3].

The data generation involves generating synthetic SELECT queries (read workload) using the parameters in Table A.1. Additionally, we generate synthetic DML statements (write workload) from a curated month-long snippet of the Amazon Redshift dataset using Stemmer’s workload generator [17]. Both synthetic workloads’ statements are verified syntactically. The workloads are also normalized by cleaning any database sharding suffixes in table names. Following that, we generate matches between the cleaned read and write workloads. The generated matches are divided into tiny workloads (also called "experiments") that consist of one SELECT statement and its corresponding DML queries. The structure of a single experiment is illustrated in Figure 3.1, while the overall workload structure is shown in Figure 3.2.

The matching between SELECT and DML queries is based on common base tables they operate on. In each experiment, the maximum number of queries is capped at 1 SELECT and 20 DML queries. The total number of queries in all generated experiments is capped at 100,000 queries. This is to ensure manageable execution times while still capturing sufficient variability.

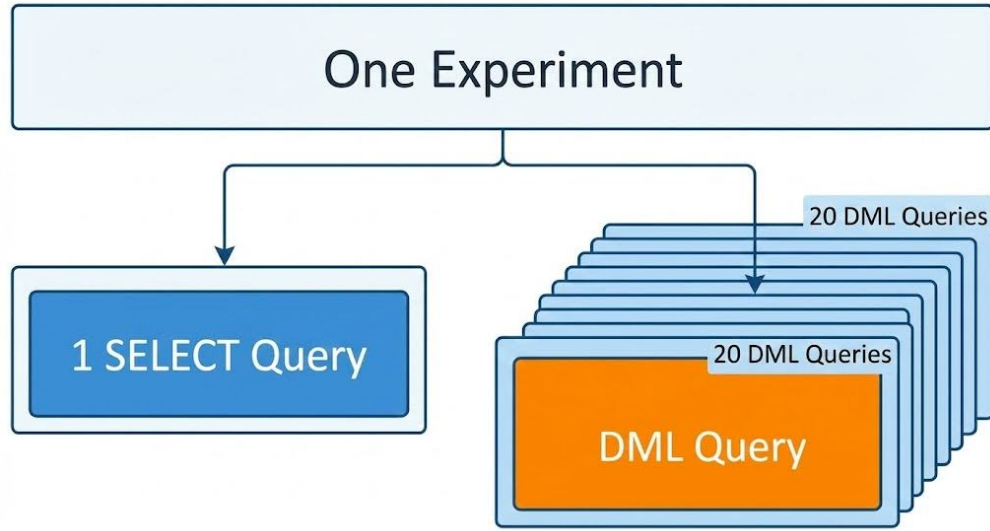


Figure 3.1.: Structure of a Single Experiment in the Workload

3.1.2. Experiment Data Collection

For each defined "experiment" workload, we execute the workload on the target database system under controlled conditions. This instrumentation is critical for capturing the ground-truth performance data.

Preparing the execution environment involves loading the schema and data, the creation of necessary indexes and loading of the necessary extensions—most importantly, the `pg_ivm` extension for IMV support. Staging tables are used to restore the database state between experiments to ensure consistency. The base tables are loaded as a 10% sample of the staging tables before each experiment execution. This is to ensure that the DML queries have data to operate on while avoiding conflicts with existing table rows. To avoid any cold start scenarios, the database is warmed up before execution. Each experiment is executed only in the IMV configuration to capture the relevant performance metrics for both reads and writes.

Each experiment execution consists of the following steps:

1. **Create the IMV for the read query:** For the read query in the experiment, we create an IMV. This sets up the necessary triggers and structures for incremental maintenance.
2. **Execute write queries and measure execution time:** Each DML query in the experiment is executed, and its execution time is measured. This captures the "trigger time overhead," which is the *view maintenance cost* incurred by the IMV triggers during writes.

This process produces a labeled dataset where each entry includes the query, its Query Execution Plan (QEP), the view configuration, the measured execution time for the DML queries and the automatically executed IMV triggers mentioned in 2.4.2.

Note that the obtained trigger execution times only measure the "wall clock" time spent executing the specific SQL statements inside the triggers. They do not include any overhead from context switching, transaction management, or other database system activities outside the trigger execution itself. The total time spent on view maintenance (including overheads) is captured in the DML total execution

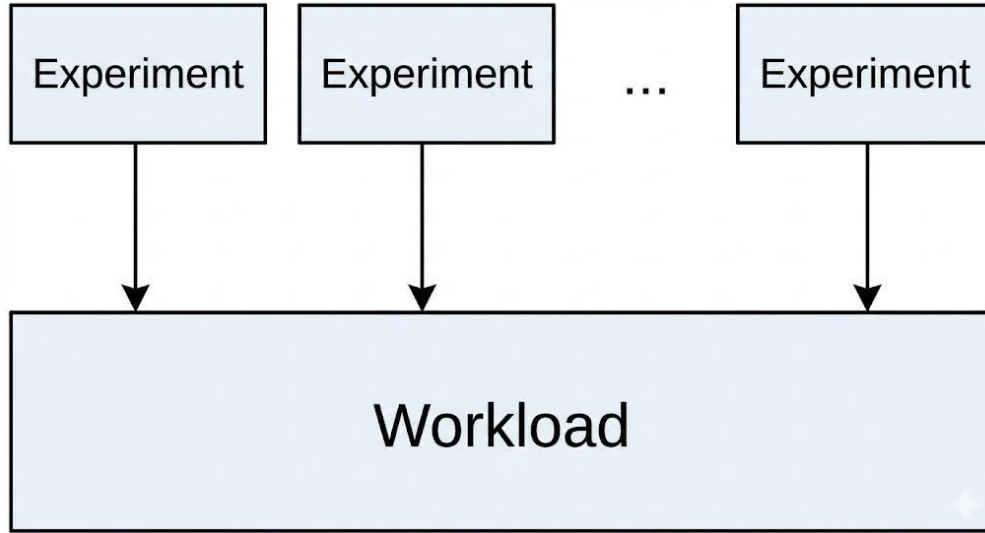


Figure 3.2.: Structure of the Entire Workload Composed of Multiple Experiments

time of the IVM triggers. This distinction is important for accurately training the cost model to predict the actual maintenance overhead.

3.1.3. Feature Extraction from Execution Plans

Once the raw performance data is collected, the next step is to parse the QEPs into numerical feature vectors. While the read-cost model utilizes the standard pipeline-centric feature extraction of the T3 system [15], the view refresh cost model requires a specialized approach to capture the overhead of IVM.

For the view refresh cost model, we extract a composite feature vector that combines two distinct sources of information:

- **Delta Features:** Extracted from the IVM trigger maintenance plans, characterizing the volume of changes (e.g., number of rows inserted/deleted, trigger execution cost).
- **Context Features:** Extracted from the view definition's QEP, characterizing the static complexity of the view (e.g., total cardinality, number of joins, total operators).
- **Interaction Features:** Scale-invariant ratios (e.g., $\frac{\text{delta rows}}{\text{view cardinality}}$) that model the relative change and allow the model to generalize across databases of varying sizes.

Prior to training, a featurized dataset is constructed by combining these feature vectors with their corresponding measured execution times. Outliers are clipped, and rows with zero execution time (indicating no actual maintenance work) are filtered out to ensure the model learns the true cost of maintenance.

3.2. Model Training

With the featurized dataset, we can train the cost models. While a pre-existing read-cost model is reused, the development of a novel *view refresh cost model* to predict the IMV maintenance overhead, which is an integral part of the view advisor.

3.2.1. Training the T3-style Cost Model

We implement the view refresh cost model as a LightGBM gradient-boosted decision-tree ensemble, an approach inspired by the T3 cost model [15]. The feature vectors extracted from the parsed trigger execution plans serve as the input (X) for this model. More specifically, for each DML QEP, the model is served the following data:

- The full set of pipeline features of the view’s underlying query, extracted from the QEP of the view query itself.
- A subset of pipeline features extracted from the QEP of the delta computation trigger associated with the IMV.
- The full set of pipeline features from the QEP of the delta apply trigger associated with the IMV.

The training pipeline was adapted to handle the specific feature subset generated from these maintenance plans augmented by contextual information from the views query plans, ensuring the model learns the costs associated with incremental updates rather than general query execution.

3.2.2. Per-Pipeline Prediction Target

A key innovation of the T3 model, which we adopt, is its training objective. Instead of predicting the total execution time of an entire pipeline, the QEP is divided into pipelines, then the model is trained to predict the **per-pipeline processing time** (t) for each pipeline. This per-pipeline time is defined as the total time spent processing tuples in that pipeline divided by the number of tuples processed (the pipeline’s cardinality). This approach allows the model to generalize better across pipelines of varying sizes and complexities.

Additionally, we train the model to minimize the mean squared error on the log-transformed per-pipeline times, which helps to handle the wide range of time values and focuses the model on relative error. To stabilize training across the wide dynamic range of per-pipeline times (which can be fractions of a nanosecond), we transform the target variable t using a negative log transformation, as described in the T3 design:

$$t' = -\log(t)$$

The model is then trained to predict this transformed value t' . At inference time, we reverse the transformation and multiply by the pipeline’s cardinality to get the final predicted pipeline cost. This per-pipeline model is the core component that allows the advisor to estimate the IMV maintenance cost for any given DML statement.

3.2.3. View Advisor Setup

The trained view refresh cost model is a key component of a larger view advisor system. The advisor is designed to recommend one of the following view configurations: IMVs, MVs or no view at all. The view advisor input is the estimate QEPs of a workload in each of the three view configurations. In case of IMV and MV, each estimate QEP of a SELECT statement contains the potential view estimated to be used. The advisor operates by estimating the effect of different view configurations on a given workload.

The setup of the view advisor involves the following steps:

1. **Workload Analysis:** The advisor takes the estimate QEPs of a given workload as input. This workload consists of a mix of read (SELECT) and DML queries. The estimate QEPs are generated for three configurations: no view, MV, and IMV.
2. **View Definitions:** A set of candidate views are created in the database beforehand by the user.
3. **Cost-Benefit Analysis:** For each candidate view, the advisor performs a cost-benefit analysis.
 - **Benefit:** The benefit is the estimated reduction in read query execution time. This is calculated by rewriting the workload queries to use the views and estimating the cost of the rewritten queries using a read-cost model.
 - **Cost:** The cost is the estimated overhead of maintaining the views. This is where our T3 view refresh cost model is used. For each DML query in the workload that affects a base table of the candidate view, the model predicts the maintenance cost.
 - **Total Score:** The total score for each view is computed as:

$$\text{Total Score} = \text{Total Read Time with View} + \text{Total View Maintenance Cost}$$

The advisor sums the predicted read times (using the read-cost model) and the predicted view maintenance costs (using the view refresh cost model) to get the total score for each view configuration.

4. **View Selection:** Based on the cost-benefit analysis, the advisor provides the following recommendations:
 - **Localized recommendations:** for each group of queries that share the same base table(s) as a candidate view, the advisor recommends whether to use the view (as MV or IMV) or not, based on which option yields the lowest total score.
 - **Global recommendation:** the advisor also provides a global recommendation across the entire workload, selecting the view configuration (no view, MV, or IMV) that minimizes the overall total score.

This is typically formulated as an optimization problem, where the goal is to maximize the total benefit under a constraint on the total maintenance cost (a "maintenance budget").

This setup allows the advisor to make informed decisions about view usage based on predicted performance impacts, leveraging the trained view refresh cost model to accurately estimate the overhead of IMV maintenance.

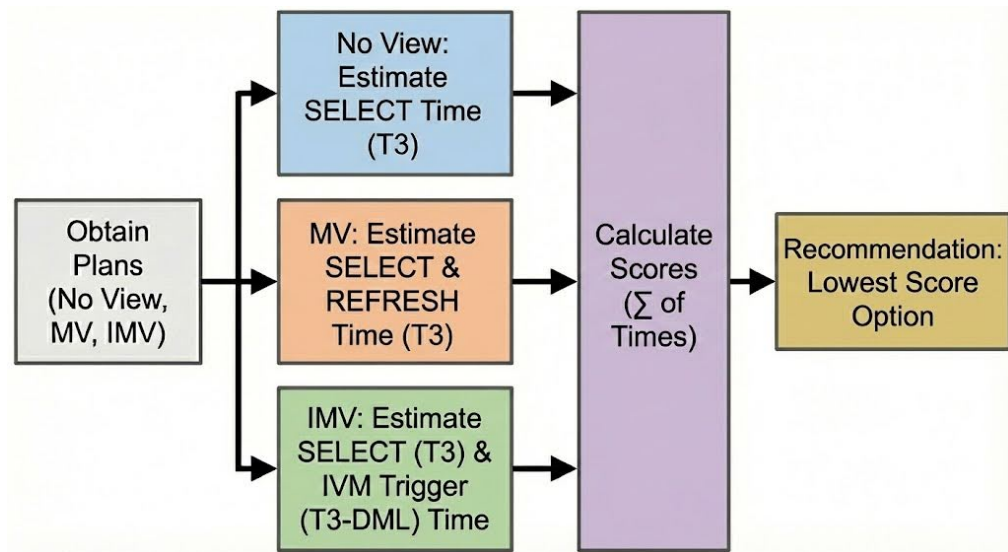


Figure 3.3.: View Advisor Cost-Benefit Analysis Flow

3.3. Summary

This chapter outlined the comprehensive methodology for developing an ML-based zero-shot view advisor. The process began with the careful curation of a labeled dataset through controlled execution of synthetic workloads, capturing both read and write performance metrics in the IMV configuration. Feature extraction techniques were employed to transform QEPs into numerical representations suitable for model training. The core of the methodology involved training a novel T3-style view refresh cost model to predict IMV maintenance overhead. Finally, the trained model was integrated into a view advisor framework capable of recommending optimal view strategies based on cost-benefit analyses. This structured approach ensures that the advisor is grounded in empirical performance data and leverages advanced machine learning techniques to make informed recommendations.

4. Evaluation

To evaluate the performance and usefulness of...

4.1. Experimental setup

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5. Conclusion and Future Work

This chapter first summarizes and concludes the contributions of this thesis by presenting the strengths behind the design concepts and concluding upon the evaluation. Following, an outlook of research challenges for future work is given.

5.1. Conclusion

Mirror the contributions given in intro.

Conclude on the evaluation Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

5.2. Future Work

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Bibliography

- [1] Rafi Ahmed et al. “Automated generation of materialized views in oracle”. In: *Proceedings of the VLDB Endowment* 13.12 (2020), pp. 3046–3058.
- [2] Mert Akdere et al. “Learning-based query performance modeling and prediction”. In: *2012 IEEE 28th International Conference on Data Engineering*. IEEE. 2012, pp. 390–401.
- [3] Anurag Gupta et al. “Amazon redshift and the case for simpler data warehouses”. In: *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 2015, pp. 1917–1923.
- [4] Ashish Gupta, Inderpal Singh Mumick, et al. “Maintenance of materialized views: Problems, techniques, and applications”. In: *IEEE Data Eng. Bull.* 18.2 (1995), pp. 3–18.
- [5] Chetan Gupta, Abhay Mehta, and Umeshwar Dayal. “PQR: Predicting query execution times for autonomous workload management”. In: *2008 International Conference on Autonomic Computing*. IEEE. 2008, pp. 13–22.
- [6] Benjamin Hilprecht and Carsten Binnig. “Zero-shot cost models for out-of-the-box learned cost prediction”. In: *arXiv preprint arXiv:2201.00561* (2022).
- [7] Benjamin Hilprecht, Carsten Binnig, and Uwe Röhm. “Learning a partitioning advisor for cloud databases”. In: *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2020, pp. 143–157.
- [8] Benjamin Hilprecht et al. “DBMS Fitting: Why should we learn what we already know?” In: *CIDR*. 2020.
- [9] Andreas Kipf et al. “Learned cardinalities: Estimating correlated joins with deep learning”. In: *arXiv preprint arXiv:1809.00677* (2018).
- [10] Jan Kossmann, Alexander Kastius, and Rainer Schlosser. “SWIRL: Selection of Workload-aware Indexes using Reinforcement Learning.” In: *EDBT*. Vol. 2. 2022, pp. 155–2.
- [11] Claude Lehmann, Pavel Sulimov, and Kurt Stockinger. “Is your learned query optimizer behaving as you expect? a machine learning perspective”. In: *arXiv preprint arXiv:2309.01551* (2023).
- [12] Sherry Listgarten and Marie-Anne Neimat. “Cost model development for a main memory database system”. In: *Real-Time Database Systems: Issues and Applications*. Springer, 1997, pp. 139–162.
- [13] Stefan Manegold, Peter Boncz, and Martin L Kersten. “Generic database cost models for hierarchical memory systems”. In: *VLDB’02: Proceedings of the 28th International Conference on Very Large Databases*. Elsevier. 2002, pp. 191–202.
- [14] Yugo Nagata and IVM Development Group. *pg_ivm: IVM (Incremental View Maintenance) implementation as a PostgreSQL extension*. https://github.com/sraoss/pg_ivm. SRA OSS LLC. 2024.

-
- [15] Maximilian Rieger and Thomas Neumann. “T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees”. In: *Proceedings of the ACM on Management of Data* 3.3 (2025), pp. 1–27.
- [16] P Griffiths Selinger et al. “Access path selection in a relational database management system”. In: *Proceedings of the 1979 ACM SIGMOD international conference on Management of data*. 1979, pp. 23–34.
- [17] Martin Stemmer. “Caching Strategies based on realistic Workloads”. Date of submission: May 22, 2025. MA thesis. Technische Universität Darmstadt, May 2025.
- [18] Ji Sun and Guoliang Li. “An end-to-end learning-based cost estimator”. In: *arXiv preprint arXiv:1906.02560* (2019).
- [19] Wentao Wu et al. “Predicting query execution time: Are optimizer cost models really unusable?” In: *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. IEEE. 2013, pp. 1081–1092.
- [20] Wentao Wu et al. “Towards predicting query execution time for concurrent and dynamic database workloads”. In: *Proceedings of the VLDB Endowment* 6.10 (2013), pp. 925–936.
- [21] Ziniu Wu et al. “Stage: Query execution time prediction in amazon redshift”. In: *Companion of the 2024 International Conference on Management of Data*. 2024, pp. 280–294.
- [22] Jiani Yang et al. “Rethinking Learned Cost Models: Why Start from Scratch?” In: *Proceedings of the ACM on Management of Data* 1.4 (2023), pp. 1–27.
- [23] Haitao Yuan et al. “Automatic view generation with deep learning and reinforcement learning”. In: *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE. 2020, pp. 1501–1512.
- [24] Yue Zhao et al. “Queryformer: A tree transformer model for query plan representation”. In: *Proceedings of the VLDB Endowment* 15.8 (2022), pp. 1658–1670.
- [25] Xuanhe Zhou et al. “Query performance prediction for concurrent queries using graph embedding”. In: *Proceedings of the VLDB Endowment* 13.9 (2020), pp. 1416–1428.

A. Experiment Parameters

The experiments conducted in the Evaluation Chapter were executed with the following DPI parameters. Note, an x indicates the parameter has been varied for the experiment.

A.1. Workload Generation Parameters

The main workload used for training and evaluation, referred to as `workload_200k_imv`, was generated using the parameters listed in Table A.1.

Table A.1.: Parameters for `workload_200k_imv`

Parameter	Value
<code>num_queries</code>	200,000
<code>min_no_joins</code>	0
<code>max_no_joins</code>	3
<code>min_no_predicates</code>	1
<code>max_no_predicates</code>	3
<code>min_no_aggregates</code>	1
<code>max_no_aggregates</code>	3
<code>max_no_group_by</code>	0
<code>max_cols_per_agg</code>	2
<code>seed</code>	1

A.2. Hardware Specifications

All workloads were executed on a machine with the type `c8220`. The key hardware and software specifications for this node are detailed in Table A.2.

A.3. IMDB Candidate View Definitions

The following SQL queries were used to define the views for the evaluation of the view advisor.

Table A.2.: Hardware and OS Specifications

Component	Specification
Machine Type	c8220
Architecture	x86_64
Processor	Intel E5-2660v2
CPU Speed	2200 MHz
CPU Sockets	2
Cores per Socket	10
Threads per Core	2
Total Logical Cores	40 (2 × 10 × 2)
Total Memory	262144 MB (256 GB)
Operating System	Ubuntu 22.04 (UBUNTU22-64-STD)

[caption=Query 1: Join on Title and AKA Title with phonetic filtering] SELECT "title"."id" AS "title_id", "aka_ttitle"."id" AS "aka_ttitle_id", "title"."kind_id", "aka_ttitle"."movie_id", "title"."production_year", "aka_ttitle"."movie_id" WHERE "aka_ttitle"."movie_id" = "title"."id" AND "aka_ttitle"."phonetic_code" BETWEEN

[caption=Query 2: Movie Companies and Types] SELECT "movie_ccompanies"."id", "movie_ccompanies"."company_itype", "company_itype"."id" WHERE "company_itype"."kind" BETWEEN 'distributors' AND 'special effects companies (production)';

[caption=Query 3: Title, Info Index, and Keywords] SELECT "title"."id", "title"."season_nr", "title"."episode_of_id", "title"."movie_iinfo_id", "movie_iinfo"."movie_id" JOIN "movie_kkeyword" ON "title"."id" = "movie_kkeyword"."movie_id" WHERE "title"."id" = "movie_kkeyword"."movie_id";

[caption=Query 4: Movie Info and Title Metadata] SELECT "movie_iinfo"."id" AS "movie_iinfo_id", "movie_iinfo"."md5sum", "title"."id" WHERE "title"."md5sum" BETWEEN '0296d2b20235be36849d015e34d80f2c' AND 'fffffec1b13';

[caption=Query 5: Cast Info ordering] SELECT "title"."id" AS "title_id", "cast_iinfo"."person_id", "cast_iinfo"."person_inr" WHERE "cast_iinfo"."nr_oder" BETWEEN 0 AND 100;

[caption=Query 6: Person Info and Name] SELECT "name"."id", "name"."name", "name"."imdb_iindex", "name"."genre_itype", "person_iinfo"."person_id" WHERE "person_iinfo"."info_type_id" BETWEEN 15 AND 39;

[caption=Query 7: Linked Movies] SELECT "movie_ilink"."id", "movie_ilink"."movie_id", "movie_ilink"."linked_mmovie_id", "movie_ilink"."link_type"."id" WHERE "movie_ilink"."linked_mmovie_id" BETWEEN 11104 AND 2524994;

[caption=Query 8: Name, AKA Name, and Person Info Join] SELECT "name"."id" AS "name_id", "aka_nname"."id" AS "aka_nname_id", "person_iinfo"."person_id" JOIN "person_iinfo" ON "name"."id" = "person_iinfo"."person_id";

[caption=Query 9: Completed Cast Status] SELECT "title"."id" AS "title_id", "complete_cast"."id" AS "complete_cast_id", "complete_cast"."movie_id" JOIN "comp_cast_type" ON "complete_cast"."subject_id" = "comp_cast_type"."id" WHERE "complete_cast"."subject_id" = "comp_cast_type"."id";

[caption=Query 10: Cast Info and Character Names] SELECT "cast_iinfo"."id" AS "cast_iinfo_id", "cast_iinfo"."movie_id", "char_name"."id" WHERE "char_name"."name_pcode_nf" BETWEEN 'H21' AND 'T5245';

[caption=Query 11: Info Types and Movie Index] SELECT "info_type"."info", "info_type"."id" FROM "info_type" JOIN "movie_iinfo_id" ON "info_type_id" WHERE "info_type"."info" BETWEEN 'LDadditional information' AND 'whenever';

[caption=Query 12: Actors and Writers] SELECT "cast_iinfo"."id", "cast_iinfo"."movie_id", "cast_iinfo"."person_id", "role_type"."id" WHERE "role_type"."role" BETWEEN 'actor' AND 'writer';